Experiment No. 9

Aim: Develop a background music player using Services.

Apparatus / Software Required :

Hardware	- Laptop(ASUS Vivobook 16x) - Minimum 8 GB RAM - AMD Ryzen 5 5600H with Radeon Graphics Processor - Internet Connection
Software	- Windows 11 Operating System - Android Studio (Quail) - Java Development Kit (JDK) - Android SDK - Android Emulator (Pixel 7) - Gradle Build System

Theory

A Service is an application component that performs long-running work without a user interface. A started service is launched with startService() / startForegroundService() and keeps running until it stops itself (stopSelf()) or is stopped. A bound service offers a client-server interface: an Activity calls bindService() and receives an IBinder through a ServiceConnection, allowing it to call methods of the service directly. This project uses both: it is started so that music continues in the background and bound so that the UI can show progress and issue commands.

Since Android 8 (API 26) background services are restricted, so an audio app must run a foreground service which displays an ongoing notification (NotificationChannel + startForeground()). Android 13 requires the runtime permission POST_NOTIFICATIONS and Android 14 requires a foreground service type (mediaPlayback) with the matching permission FOREGROUND_SERVICE_MEDIA_PLAYBACK.

Implementation Steps

1. Create an Android project using Java, package com.example.musicservice, and Min SDK 24.
2. Add an audio file named sample_music.mp3 to the res/raw folder.
3. Design the UI with a song title, status, SeekBar, time display, and Play/Pause/Stop buttons.
4. Declare the required permissions and MusicService in AndroidManifest.xml.
5. Implement MusicService using MediaPlayer, a foreground notification, action buttons, and a Binder.

6. Use MainActivity to control playback, seek through the song, and update progress every 500 ms.
7. Test the app by playing music and pressing Home. Music should continue playing in the background, and notification buttons should control playback.

Files Used / Code Changes

File	Purpose / Change
MusicService.java	Foreground service, MediaPlayer, notification, binder, playback methods.
MainActivity.java	UI, service binding, SeekBar and permission request.
activity_main.xml	Player layout.
res/raw/sample_music.wav	Generated sample melody (replaceable by any audio file with the same name).
drawable/ic_music_note.xml	Notification and UI icon.
AndroidManifest.xml	Service declaration and permissions.
strings.xml, colors.xml, themes.xml, build.gradle files	Resources and Gradle configuration.

Components Used

Component	Use in this project
Service (started + bound, foreground)	Plays music independent of the UI.
Binder / ServiceConnection	Activity-service communication.
MediaPlayer	Audio playback.
Notification, NotificationChannel, PendingIntent	Foreground notification with actions.
Handler + Runnable	Periodic UI refresh.
SeekBar, Button, TextView	Player controls.
Activity Result API	Notification permission request.

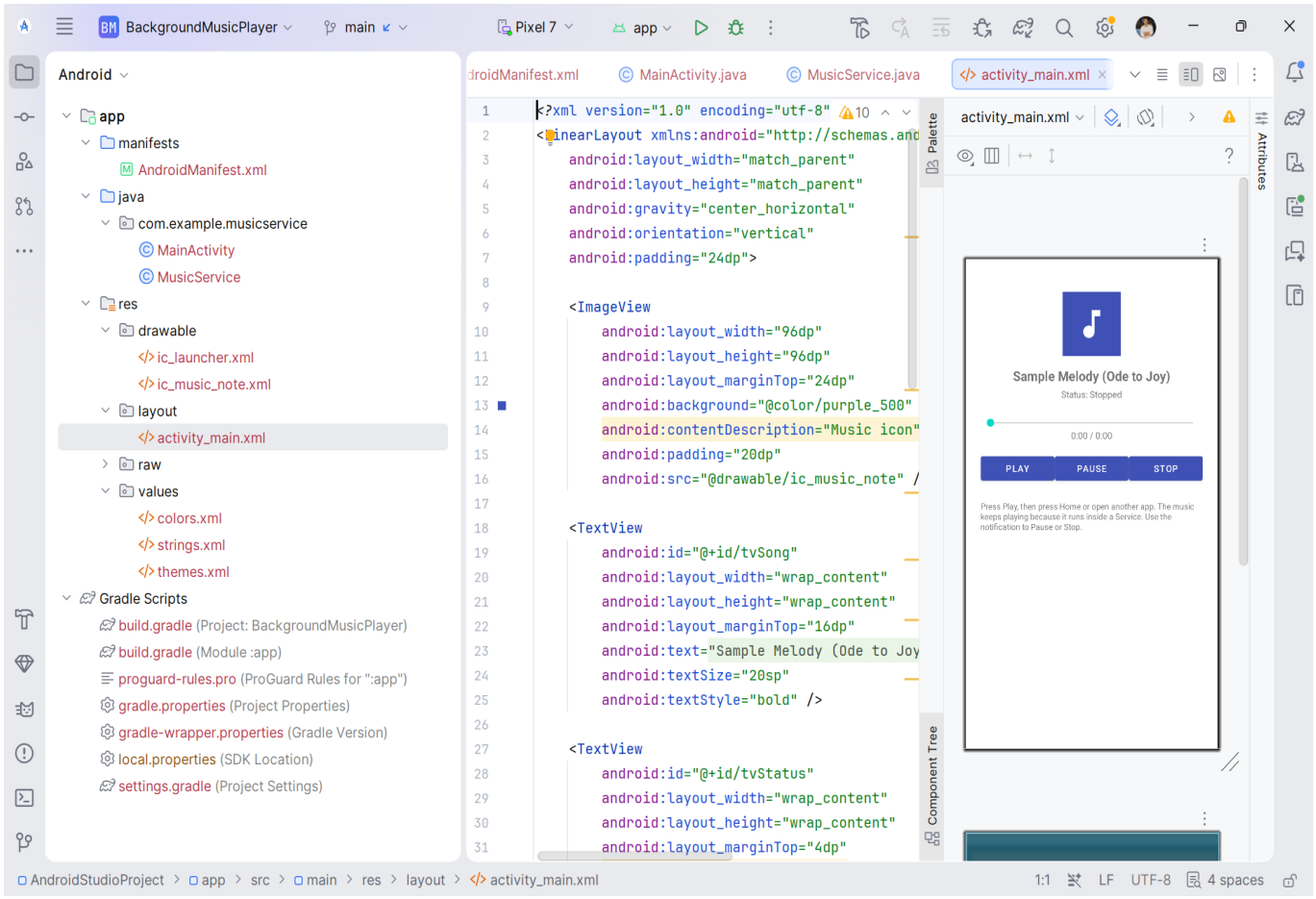
Important XML Attributes Used

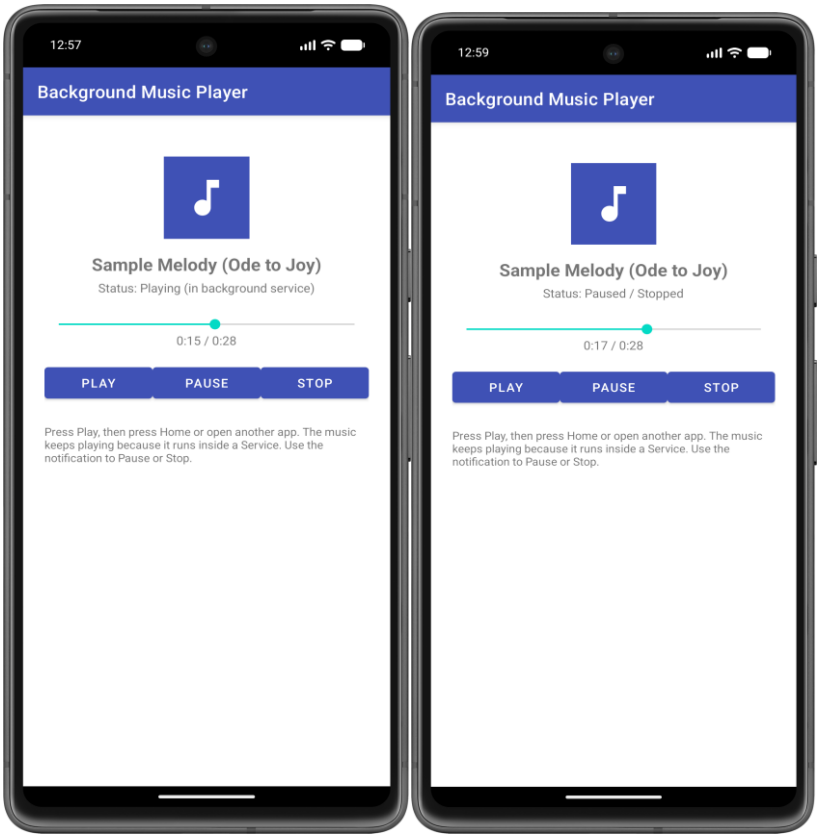
Attribute	Meaning
service android:foregroundServiceType="mediaPlayback"	Declares the type of foreground service (Android 14+).
service android:exported="false"	Only this app can start the service.
uses-permission FOREGROUND_SERVICE, FOREGROUND_SERVICE_MEDIA_PLAYBACK	Allow foreground media service.
uses-permission POST_NOTIFICATIONS	Allow showing notifications (Android 13+).
android:launchMode="singleTop"	Re-use the open activity when opened from the notification.
SeekBar (progress / max)	Shows and sets playback position.
android:layout_weight="1"	Equal-width buttons.

Important Java Methods Used

Method / Class	Purpose
startForegroundService(Intent)	Starts the service (must call startForeground within 5 s).
bindService() / unbindService()	Connect / disconnect Activity and service.
onCreate(), onStartCommand(), onBind(), onDestroy()	Service lifecycle callbacks.
startForeground(), stopForeground(), stopSelf()	Foreground state and stopping.
MediaPlayer.create(), start(), pause(), seekTo(), release()	Playback control.
getCurrentPosition(), getDuration(), isPlaying()	Playback state.
NotificationCompat.Builder.addAction()	Notification buttons.
PendingIntent.getService()	Notification button sends action to service.
Handler.postDelayed()	Update UI every 500 ms.

Output





Conclusion

The experiment shows how a started and bound foreground Service can own a MediaPlayer to play music in the background, how the Activity controls it through a Binder and how a notification provides controls while the app is closed. The project is complete as source code; run it on a device or emulator to verify the behaviour and capture the screenshots.

C (3)	A (3)	P (3)	T (9)	Sign